March 29, 2026 9:43 PM

1. Use case description

Book Store Application Report

1. Use Case Description

Use Case Name

Customer Redeems Points and Buys Books

Primary Actor

Customer

Description

This use case describes how a customer selects books, redeems accumulated points, and completes a purchase.

Preconditions

* The customer must be registered
* The customer must be logged in
* Books must be available in the system
* The customer has at least one book selected

Postconditions

* The purchase is completed
* Customer points are updated
* Customer status (Silver or Gold) is updated

Main Flow

1. The customer logs into the system
2. The system displays the list of available books
3. The customer selects one or more books
4. The customer clicks the **“Redeem points and Buy”** button
5. The system calculates the total cost of selected books
6. The system calculates the discount: every 100 points = $1 off the total cost
7. The system ensures the total cost is not less than zero
8. The system deducts the redeemed points from the customer's balance and earns new points on the final amount paid (10 points per $1 spent)
9. The system updates the customer’s status if needed
10. The system displays total cost, points, and status

Alternative Flow

* If the customer does not have enough points, only partial redemption is applied
* If points fully cover the cost, the total cost becomes zero
* If no books are selected, the system does nothing

2. Rationale behind using the State Design Pattern

The State Design Pattern was selected to manage the behavior of the Customer class based on their loyalty points.

1. Encapsulation of Varying Behavior
The application requires different logic for Silver and Gold members (e.g., different point accumulation or status names). By using the State Pattern, these rules are encapsulated in SilverState and GoldState classes. This prevents the Customer class from becoming bloated with "if-else" or "switch" statements every time a purchase or redemption occurs.

2. Simplified State Transitions
In the provided implementation, the transition logic is handled internally by the states. For example, the SilverState checks if points have reached 1000 and triggers the change to GoldState automatically. This allows the Customer object to change its behavior at runtime without the main system needing to monitor thresholds constantly.

3. Improved Maintainability and Scalability
The State Design Pattern follows the open/closed principle. For instance, if the bookstore wants to add another membership level called platinum in the future, it simply needs to add another class that extends the StatusState interface. The code in the Customer class and the BookStoreSystem class does not need to be touched. Thus, the chance of introducing bugs into the code is minimized.

4. Integration with Data Persistence
The pattern integrates nicely with our FileManager class. When reading data from the file customers.txt, it reads the point value and then assigns the correct initial state object, SilverState or GoldState. This ensures that our object-oriented approach is maintained even if the program is restarted.